8.5 Data Types and Coding Style Guide

Data Types

Each port of FreeRTOS has a unique portmacro.h header file that contains (amongst other things) definitions for two special data types, portTickType and portBASE_TYPE. These data types are described in Table 30.

Table 30. Special data types used by FreeRTOS

Macro or typedef used	Actual type
portTickType	This is used to store the tick count value and to specify block times. portTickType can be either an unsigned 16-bit type or an unsigned 32-bit type, depending on the setting of configUSE_16_BIT_TICKS within FreeRTOSConfig.h. Using a 16-bit type can greatly improve efficiency on 8-bit and 16-bit architectures, but severely limits the maximum block period that can be specified. There is no reason to use a 16-bit type on a 32-bit architecture, so configUSE_16_BIT_TICKS should be set to 0.
portBASE_TYPE	This is always defined as the most efficient data type for the architecture. Typically, this is a 32-bit type on a 32-bit architecture, a 16-bit type on a 16-bit architecture, and an 8-bit type on an 8-bit architecture. portBASE_TYPE is generally used for return types that can take only a very limited range of values, and for Booleans. Cortex-M3 ports define portBASE_TYPE as type ‘long’.

Some compilers make all unqualified char variables unsigned, while others make them signed. For this reason, the FreeRTOS source code explicitly qualifies every use of char with either ‘signed’ or ‘unsigned’.

Plain int types are never used—only long and short.

Variable Names

Variables are prefixed with their type: 'c' for char, 's' for short, 'l' for long, and 'x' for portBASE_TYPE and any other type (structures, task handles, queue handles, etc.).

If a variable is unsigned, it is also prefixed with a 'u'. If a variable is a pointer, it is also prefixed with a 'p'. Therefore, a variable of type unsigned char will be prefixed with 'uc', and a variable of type pointer to char will be prefixed with 'pc'.

Function Names

Functions are prefixed with both the type they return and the file they are defined within. For example:

- vTaskPrioritySet() returns a void and is defined within task.c.
- xQueueReceive() returns a variable of type portBASE_TYPE and is defined within queue.c.
- vSemaphoreCreateBinary() returns a void and is defined within semphr.h.

File scope (private) functions are prefixed with 'prv'.

Formatting

One tab is always set to equal four spaces.

Macro Names

Most macros are written in upper case and prefixed with lower case letters that indicate where the macro is defined. Table 31 provides a list of prefixes.

Table 31. Macro prefixes

Prefix	Location of macro definition
port (for example, portMAX_DELAY)	portable.h
task (for example, taskENTER_CRITICAL())	task.h
pd (for example, pdTRUE)	projdefs.h
config (for example, configUSE_PREEMPTION)	FreeRTOSConfig.h
err (for example, errQUEUE_FULL)	projdefs.h

Note that the semaphore API is written almost entirely as a set of macros, but follows the function naming convention, rather than the macro naming convention.

The macros defined in Table 32 are used throughout the FreeRTOS source code.

Table 32. Common macro definitions

Macro	Value
pdTRUE	1
pdFALSE	0
pdPASS	1
pdFAIL	0

Rationale for Excessive Type Casting

The FreeRTOS source code can be compiled with many different compilers, all of which differ in how and when they generate warnings. In particular, different compilers want casting to be used in different ways. As a result, the FreeRTOS source code contains more type casting than would normally be warranted.